Project Report: Rescue Mission Order

(Traveling Salesperson Variant)

Team Members:
Michael Ayed
Arwa Mohamed

Department: Artificial Intelligence (AI)
Faculty: Faculty of Computers and Information, Assiut University

Date: May 2026

1. The Environment (State Space Definition)

The environment simulates a real-world restricted search space. It is represented as a 10×10 2D array (Grid) using the `NumPy` library.

- **Grid Logic:** `0` represents a traversable path, and `1` represents an impassable wall/obstacle.
- **Agents:** The robot starts at base coordinate $(0, 0)$. Five victims are scattered across specific coordinates.
- **Data Handling:** The robot and victims are combined into a single `locations` array to be seamlessly indexed (0 to 5) during the pathfinding and optimization phases.

2. Search Algorithm: Pathfinding and Distance

To calculate the true physical distance between any two nodes avoiding walls, we implemented a **Breadth-First Search (BFS)** algorithm.

Why BFS over DFS? (A Brief Comparison)

- **BFS (Breadth-First Search):** Explores the grid level-by-level equally in all directions. In an unweighted grid (where every step costs exactly 1), BFS is **mathematically guaranteed to find the absolute shortest path**. This is critical for our Distance Matrix; we need the absolute minimum steps between victims, not just "any" path.
- **DFS (Depth-First Search):** Explores as deep as possible down one path before backtracking. While DFS uses less memory, it only finds *a path*, which is rarely the shortest. Using DFS in a rescue mission would result in highly inefficient and misleading distances, ruining the Genetic Algorithm's calculations. Hence, BFS was the optimal and only logical choice.

3. Genetic Algorithm (GA) Optimization Design

Because the Traveling Salesperson Problem (TSP) is NP-Hard, calculating all $O(n!)$ permutations using brute force is unscalable. We used a Genetic Algorithm to find the optimal visitation sequence quickly.

3.1. Chromosome Representation

The chromosome represents the **order of visits**. It is an integer permutation of the numbers 1 to 5 (representing victims V_1 to V_5). Example: $C = [2, 1, 3, 5, 4]$.

3.2. Fitness Function

What is it? The fitness function calculates the total true steps required to execute the sequence proposed by the chromosome, using the Distance Matrix generated by BFS.
Why this function? The objective is to minimize travel time. The function rewards efficient, closely-grouped sequences with a low total step count and penalizes erratic, back-and-forth sequences with a high step count.

Implementation Logic (Python Code):

```
def calculate_fitness(chromosome):
    # Distance from Robot base (Index 0) to the 1st victim
    total = dist_matrix[0][chromosome[0]]

    # Calculate step distances between consecutive victims
    for i in range(len(chromosome)-1):
        total += dist_matrix[chromosome[i]][chromosome[i+1]]

    return total
```

3.3. Selection Algorithm

What is it? We used **Tournament Selection** ($k = 2$). It randomly picks two chromosomes and selects the one with the better (lower) fitness score.
Why is it the best fit? Unlike Roulette Wheel selection, which struggles when fitness scores are very close to each other, Tournament Selection maintains a strong and consistent *selection pressure*. It is computationally fast and effectively balances exploration (randomness) with exploitation (choosing the best). We also implemented **Elitism** (`best.copy()`) to strictly preserve the absolute best solution across generations.

3.4. Crossover Algorithm

What is it? We implemented **Order Crossover (OX)**.
Why is it the best fit? Standard crossovers (like Single-Point) destroy TSP chromosomes because they create duplicates or delete victims (e.g., visiting V_1 twice and never visiting V_3). Order Crossover perfectly preserves the validity of the permutation by copying a sub-segment from Parent 1 and filling the remaining slots sequentially from Parent 2.

3.5. Mutation Algorithm

What is it? We used **Swap Mutation** with a rate of 20% (`MUTATION_RATE = 0.2`).
Why is it the best fit? It randomly selects two victims in the sequence and swaps their positions. This guarantees the chromosome remains a valid permutation while introducing enough genetic diversity to prevent the algorithm from getting stuck in a local optimum.

3.6. Termination Phase

The algorithm concludes the evolutionary process when it reaches a state where further searching is no longer efficient. We defined three logical criteria for our Termination Phase:

1. **Fixed Iterations:** The algorithm stops after reaching a predefined generation limit (`GEN_COUNT = 200`). This provides a predictable execution time and is more than sufficient given our search space size.

2. **Convergence:** If the population's fitness remains stagnant and the best solution does not improve for a specific number of generations, the algorithm concludes that it has stabilized at the global optimum.

3. **Target Threshold Achievement:** The algorithm can terminate early upon reaching a predefined optimal fitness target that satisfies the required rescue time constraints.

4. Visualization Module

What is used? We utilized the `Matplotlib` library to construct a 2D map of the environment.
What is the benefit of this image? The visualization proves that our logic is flawlessly bridging the gap between abstract sequences and physical reality. Instead of drawing theoretical straight lines between victims, the lines in the visualization trace the **exact cell-by-cell path generated by the BFS algorithm**. It visually verifies to the observer that:

1. The robot successfully avoids all black wall obstacles.

2. The GA's optimal sequence is practically executable in the physical environment.

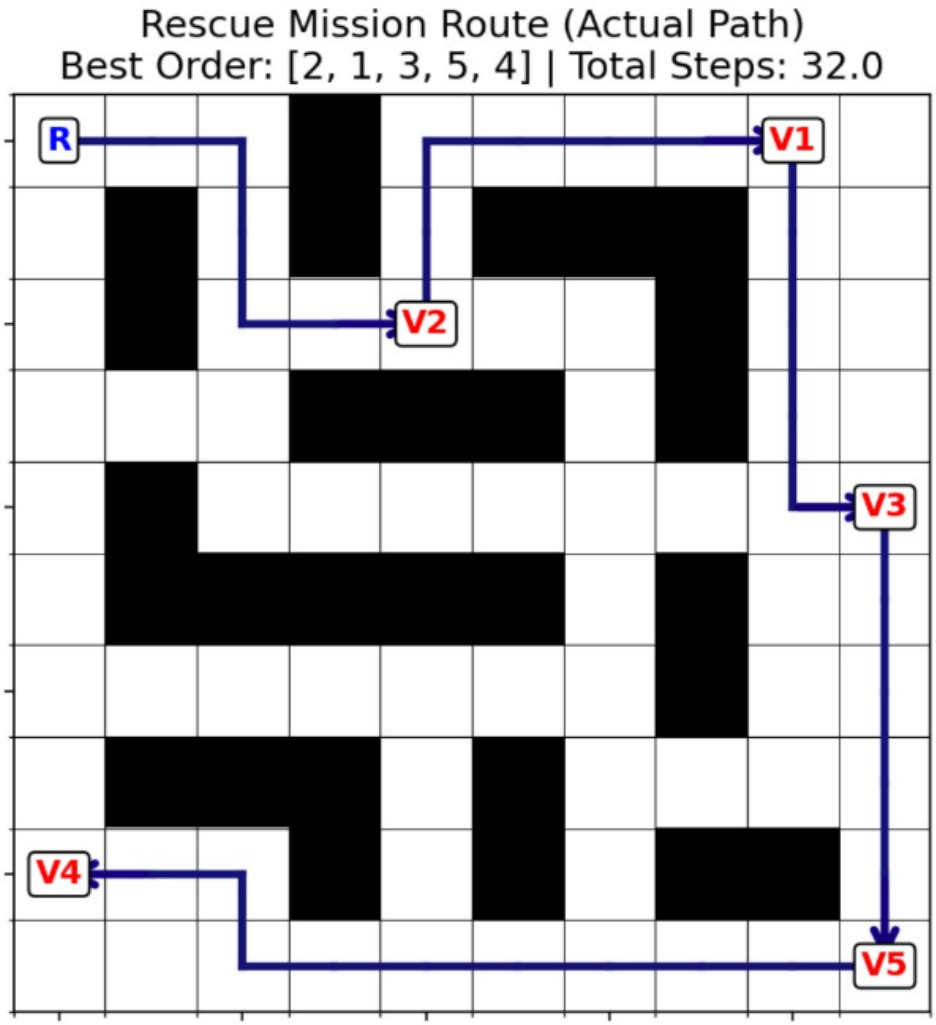


Figure 1: Matplotlib visualization of the final optimized rescue route. The lines trace the exact step-by-step physical path computed by the BFS algorithm to successfully navigate around wall obstacles. The robot (R) executes the optimal victim sequence derived by the Genetic Algorithm, completing the entire mission efficiently.

5. Team Roles and Contributions

The successful implementation of this project was evenly distributed, combining algorithmic optimization with spatial logic and visualization. The specific contributions are as follows:

1. **Michael Ayed:** Developed the physical 2D grid environment and the BFS pathfinding module. Successfully generated the true Distance Matrix, implemented the path reconstruction logic (`bfs_path`), and handled the Matplotlib visualization module to dynamically render the exact cell-by-cell physical route avoiding walls.

2. **Arwa Mohamed:** Handled the Genetic Algorithm Optimization logic. Formulated the mathematical implementation of the Fitness Function, and programmed the GA lifecycle including Tournament Selection, Order Crossover (OX), Swap Mutation, and Elitism tracking.